

EXTENSIONES FUNCIONALES DE EJEMPLO

- Clínica Veterinaria:

O ControlCheck 1 G1 -----

En este ejercicio, añadiremos la funcionalidad de gestión de enfermedades, síntomas y tratamientos médicos. Concretamente, se proporciona una clase “Disease” que representa a las enfermedades que pueden desarrollar las mascotas, que se relaciona con el tipo de mascotas que pueden sufrirlas. Además, tendremos las clases “Symptom” y “Treatment” que representan a los síntomas que pueden aparecer a consecuencia de una enfermedad y los tratamientos recomendados para cada enfermedad respectivamente. Además, se ha creado una relación que indica qué síntomas son susceptibles de presentarse para una enfermedad llamada “includes”, para ayudar a los veterinarios a realizar diagnósticos más precisos. El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:

classDiagram

direction LR

```
class NamedEntity {  
    +String name  
}
```

```
class Person {  
    +String firstName  
    +String lastName  
}
```

```
class Vet {  
}
```

```
class Owner {
```

```
+String address
+String city
}
```

```
class Specialty {
}
```

```
class Pet {
    +Date birthDate
    +Double weight
}
```

```
class PetType {
}
```

```
class Visit {
    +Date date
    +String description
}
```

```
class Disease {
    +String description
    +Integer severity
}
```

```
class Symptom {
    +String virulence
}
```

```

class Treatment {
    +Integer baseDose
    +Integer shockDose
    +Integer maxDose
}

```

%% --- Herencia ---

```

NamedEntity <|-- Pet
NamedEntity <|-- Specialty
NamedEntity <|-- Symptom
NamedEntity <|-- Disease
NamedEntity <|-- Treatment

```

```

Person <|-- Vet

```

```

Person <|-- Owner

```

%% --- Asociaciones y Relaciones ---

```

Owner "1" -- "0..n" Pet : owns >
Vet "0..n" --> "0..1" Specialty
Vet "1" <-- "0..n" Visit
Pet "0..n" --> "1" PetType
Pet "1" <-- "0..n" Visit
Visit "0..n" --> "0..1" Disease : <font color=blue>diagnosed</font>
Visit "0..n" --> "0..n" Symptom : <font color=red>present</font>
PetType "0..n" <-- "0..n" Disease : <font color=blue>can
develop</font>
Disease "1..n" <-- "0..n" Symptom : <font color=red>includes</font>
Disease "1..n" <-- "0..n" Treatment : <font color=red>recommended
for</font>

```

%% Style

style Pet stroke:blue,color:blue

style PetType stroke:blue,color:blue

style Disease stroke:blue,color:blue

style Visit stroke:blue,color:blue

style Symptom stroke:red,color:red

style Treatment stroke:red,color:red

linkStyle 7 stroke:blue

linkStyle 6 stroke:blue

Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control de laboratorio. Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias.

O ControlCheck 1 G2

En este ejercicio, añadiremos la funcionalidad de gestión de cirugías a las mascotas. En concreto, tendremos la clase “Surgery”, que representa la realización de una cirugía a una mascota por un equipo de veterinarios, y la clase “OperatingRoom”, que representa las distintas salas de operaciones disponibles en la clínica para realizar la cirugía. Cada sala de operaciones está preparada para tratar a un determinado tipo de cirugía concreto, representado con la clase “SurgeryType”. Por ejemplo, podemos tener las salas 1 y 2 preparadas para cirugía de tejidos blandos y la sala 3 preparada para cirugía oftálmica. Además, la clase SurgeryType se relaciona con aquellos PetType a los que ese tipo de cirugía puede aplicar. El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:

```
classDiagram
```

```
direction LR
```

```
class NamedEntity {  
    +String name  
}
```

```
class Person {  
    +String firstName  
    +String lastName  
}
```

```
class Vet {  
}
```

```
class Owner {  
    +String address  
    +String city  
}
```

```
class Specialty {  
}
```

```
class Pet {  
    +Date birthDate  
    +Double weight  
}
```

```
class PetType {
```

```
}
```

```
class Visit {  
    +Date date  
    +String description  
}
```

```
class Surgery {  
    +String description  
    +LocalDate date  
}
```

```
class SurgeryType {  
}
```

```
class OperatingRoom {  
    +int size  
    +String sterilizationLevel  
}
```

```
%% --- Herencia ---
```

```
NamedEntity <|-- Pet
```

```
NamedEntity <|-- Specialty
```

```
NamedEntity <|-- SurgeryType
```

```
NamedEntity <|-- Surgery
```

```
NamedEntity <|-- OperatingRoom
```

```
Person <|-- Vet
```

```
Person <|-- Owner
```

%% --- Asociaciones y Relaciones ---

Owner "1" -- "0..n" Pet : owns >

Vet "0..n" --> "0..1" Specialty

Pet "0..n" --> "1" PetType

Visit "0..n" --> "1" Pet

Visit "0..n" --> "1" Vet

Visit "0..n" --> "0..1" SurgeryType : recommends

SurgeryType "0..n" --> "1..n" PetType : can be applied to

Surgery "0..n" --> "1" Pet : pet

Surgery "1..n" --> "1" SurgeryType : type

Surgery "0..n" --> "1" OperatingRoom : room

OperatingRoom "0..n" --> "1..n" SurgeryType : valid for

%% Style style SurgeryType

stroke:blue,color:blue style

Surgery stroke:red,color:red style

OperatingRoom

stroke:red,color:red

Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control de laboratorio. Commented [IE1]: Cambiar en el repositorio el PDF con la definición del ejercicio. Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias.

o ControlCheck 1 G3 -----

En este ejercicio, añadiremos la funcionalidad de gestión de cursos de especialización sobre mascotas para los veterinarios de las clínicas. Concretamente, se proporciona una clase “Course” que representa los cursos disponibles, relacionados con los tipos de mascota sobre los que versan y el veterinario que actúa como profesor. Además, tendremos las clases “CourseAttendance” y “CoursePayment” que registran la información asociada con los veterinarios que asisten al curso como alumnos y el pago de las tasas correspondientes, dado que el profesor no trabaja gratis. El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:

classDiagram

direction LR

```
class Person {  
    +String firstName  
    +String lastName  
}
```

```
class Vet {  
}
```

```
class Owner {  
    +String address  
    +String city  
}
```

```
class Speciality {  
}
```

```
class Pet {
```



```
+Date birthDate
+Double weight
}
```

```
class PetType {
}
```

```
class CourseAttendance {
    +LocalDate registeredOn
    +Integer grade
}
```

```
class CoursePayment {
+LocalDate paidOn
    +Double amount
}
```

```
class Course {
    +String description
    +Integer maxAttendants
    +Double price
}
```

%% --- Herencia ---

Person <|-- Vet

Person <|-- Owner

%% --- Asociaciones y Relaciones ---

Owner "1" -- "0..n" Pet : owns >

```

Vet "0..n" --> "0..1" Speciality
Pet "0..n" --> "1" PetType
PetType "0..n" <--> "0..n" Course : <font color=blue>< is
about</font>
Course "0..n" --> "0..1" Speciality : <font color=blue>is about
></font>
Course "0..n" --> "1" Vet : <font color=blue>< taught by</font>
CourseAttendance "0..n" <--> "1"Course
CourseAttendance "1" --> "0..n" CoursePayment
CourseAttendance "0..n" --> "1" Vet : <font color=red>attended by
></font>

```

%% Style

```

style PetType stroke:blue,color:blue
style Vet stroke:blue,color:blue
style Course stroke:blue,color:blue

style CourseAttendance stroke:red,color:red
style CoursePayment stroke:red,color:red

linkStyle 8 stroke:blue
linkStyle 9 stroke:blue

```

Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control de laboratorio. Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias.

En este ejercicio, añadiremos la funcionalidad de gestión de enfermedades, síntomas y medicamentos. Concretamente, se proporciona una clase “Disease” que representa a las enfermedades que pueden desarrollar las mascotas, que se relaciona con el tipo de mascotas que pueden sufrirlas. Esta clase, además de la descripción y severidad de la enfermedad, presenta los atributos de contagiosidad y de periodo de incubación de la dolencia. Además, tendremos las clases “Symptom” y “Medicine” que representan a los síntomas que pueden aparecer a consecuencia de una enfermedad y los medicamentos recomendados para cada enfermedad respectivamente. Además, se ha creado una relación que indica qué síntomas son susceptibles de presentarse para una enfermedad llamada “includedDiseases”, y otra relación para indicar qué síntomas excluyen que se trate de ciertas enfermedades llamada “excludedDiseases”, para ayudar a los veterinarios a realizar diagnósticos más precisos. Por otro lado, tenemos una relación denominada “prescribedfor” para indicar la medicación recetada para la cura de la afección de la mascota. El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:

classDiagram

direction LR

```
class NamedEntity {  
    +String name  
}
```

```
class Person {  
    +String firstName  
    +String lastName  
}
```

```
class Vet {
```

```
}
```

```
class Owner {  
    +String address  
    +String city  
}
```

```
class Specialty {  
}
```

```
class Pet {  
    +Date birthDate  
    +Double weight  
}
```

```
class PetType {  
}
```

```
class Visit {  
    +Date date  
    +String description  
}
```

```
class Disease {  
    +String description  
    +Integer severity  
    +Boolean contagious  
    +Integer incubationPeriod  
}
```

```
class Symptom {  
    +String description  
}
```

```
class Medicine {  
    +String description  
    +Integer medication  
    +LocalDate startDate  
    +LocalDate endDate  
}
```

%% --- Herencia ---

```
NamedEntity <|-- Pet  
NamedEntity <|-- Speciality  
NamedEntity <|-- Disease  
NamedEntity <|-- Medicine  
NamedEntity <|-- Symptom  
Person <|-- Vet  
Person <|-- Owner
```

%% --- Asociaciones y Relaciones ---

```
Owner "1" -- "0..n" Pet : owns >  
Vet "0..n" --> "0..1" Specialty  
Vet "1" <-- "0..n" Visit  
Pet "0..n" --> "1" PetType  
Pet "1" <-- "0..n" Visit  
Visit "0..n" --> "0..1" Disease : <font color=blue>diagnosed</font>  
Visit "0..n" --> "0..n" Symptom : <font color=red>present</font>
```

PetType "0..n" <-- "0..n" Disease : can
develop

Disease "1..n" <-- "0..n" Symptom : <font
color=red>includesDiseases

Disease "0..n" <-- "0..n" Symptom : <font
color=red>excludesDiseases Speciality -- Disease

Speciality -- Medicine

Speciality -- Symptom

Disease "1..n" <-- "0..n" Medicine : <font
color=red>prescribedfor

%% Style

style Pet stroke:blue,color:blue

style PetType stroke:blue,color:blue

style Disease stroke:blue,color:blue

style Visit stroke:blue,color:blue

style Symptom stroke:red,color:red

style Medicine stroke:red,color:red

Las clases para las que realizaremos el mapeo como entidades JPA se han señalado en rojo. Las clases azules son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control. Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias.

En este ejercicio, añadiremos la funcionalidad de ofrecer servicios de belleza a las mascotas (en inglés pet grooming) de forma combinada junto a las visitas a los veterinarios. Esto lo modelaremos con una clase llamada GroomingType que incluye una descripción. Ejemplos de GroomingType pueden ser “cepillado”, “baño”, “baño con masaje” o “corte de uñas”. Los GroomingType se agrupan en GroomingPackage, que incluyen, además del nombre, una descripción y su coste. Los GroomingType están disponibles para un determinado tipo de mascotas (PetType). Por ejemplo, el “corte de uñas” está disponibles para gatos y perros, pero no para serpientes. Un GroomingPackage puede constar únicamente de un GroomingType como “baño con masaje” o de varios GroomingType como “cepillado + baño + corte de uñas”. Finalmente, los usuarios compran un Coupon de un determinado GroomingPackage que van consumiendo (GroomingConsumed) en una o varias visitas. Los Coupon además tiene una fecha de validez determinada por los atributos startDate (inicio) y expirationDate (caducidad). Cuando un servicio de belleza es consumido (GroomingConsumed) en una visita, se debe indicar el nombre del esteticista de mascotas que realizó el servicio, el número de minutos invertidos y, además, es posible agregar un comentario sobre el servicio.

classDiagram

direction LR

```
class NamedEntity {  
    +String name  
}
```

```
class BasedEntity {  
    +Integer id  
}
```

```
class Person {
```

```
    +String firstName  
    +String lastName  
}
```

```
class Vet {  
}
```

```
class Owner {  
    +String address  
    +String city  
}
```

```
class Specialty {  
}
```

```
class Pet {  
    +Date birthDate  
    +Double weight  
}
```

```
class PetType {  
}
```

```
class Visit {  
    +Date date  
    +String description  
}
```

```
class GroomingPackage {
```



```
+String description
+Double cost
}
```

```
class GroomingType {      +String description
}
```

```
class GroomingConsumed {
    +String petBeautician
    +Integer minutes
    +String comment
}
```

```
class Coupon {
    +LocalDate startDate
    +LocalDate expiryDate
}
```

```
%% --- Herencia ---
```

```
BasedEntity <|-- NamedEntity
```

```
NamedEntity <|-- Pet
```

```
NamedEntity <|-- Person
```

```
NamedEntity <|-- Specialty
```

```
NamedEntity <|-- PetType
```

```
NamedEntity <|-- GroomingType
```

```
NamedEntity <|-- GroomingPackage
```

```
Person <|-- Vet
```

```
Person <|-- Owner
```

```

Vet "0..n" --> "0..1" Specialty
Vet "1" <-- "0..n" Visit
Owner "1" -- "0..n" Pet : owns >
Pet "" --> "" PetType
Pet "1" -- "0..n" Visit
PetType "1..n" <-- "0..n" GroomingType : <font
color=blue>availableTo</font>
GroomingPackage "0..n" --> "1..n" GroomingType : <font
color=blue>contents</font>
GroomingPackage "1" <-- "0..n" Coupon : <font
color=red>groomingPackage</font>
GroomingConsumed "0..n" --> "1..n" GroomingType : <font
color=red>typeConsumed</font>
GroomingConsumed "0..n" --> "1" Coupon: <font
color=red>consumed</font>
GroomingConsumed "0..n" --> "1" Visit: <font
color=red>appliedDuring</font>

```

%% Style

```

style GroomingType stroke:blue,color:blue
style GroomingPackage stroke:blue,color:blue

```

```

style GroomingConsumed stroke:red,color:red
style Coupon stroke:red,color:red

```

Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control de laboratorio. Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias.

o ControlCheck 2 G3 -----

En este ejercicio, añadiremos la funcionalidad de gestión de alergias de las mascotas a principios activos de los medicamentos prescritos en las visitas de las mascotas. Concretamente, se proporciona una clase que representa los medicamentos disponibles que está relacionada con otra clase que representa los principios activos que pueden contener. Como objetivo, tendremos que completar las clases “Allergy” que representa las alergias de las mascotas a los distintos principios activos, así como “Prescription” que representa la prescripción de medicamentos obligatorios y opcionales para una mascota en una visita y que obviamente tendrá que tener en cuenta las alergias de dicha mascota. El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:

classDiagram

direction LR

```
class NamedEntity {  
    +String name  
}
```

```
class Person {  
    +String firstName  
    +String lastName  
}
```

```
class Vet {  
}
```

```
class Owner {  
    +String address
```

```
+String city  
}
```

```
class Pet {  
    +Date birthDate  
    +Double weight  
}
```

```
class Visit {  
    +Date date  
    +String description  
}
```

```
class Allergy {  
    +Integer[1..10] level  
    +LocalDate diagnosisDate  
    +String information  
}
```

```
class ActivePrinciple {  
    +Integer maxDailyDose  
}
```

```
class Medication {  
+String brand  
    +Integer dosePerUnit  
    +Integer units  
}
```

```

class Prescription {
+Integer dosage
    +LocalDate treatmentStart    +LocalDate treatmentEnd
    +Integer hoursBetweenDosage
    +String information
}

```

%% --- Herencia ---

NamedEntity <|-- Pet

NamedEntity <|-- ActivePrinciple

NamedEntity <|-- Medication

Person <|-- Vet

Person <|-- Owner

Vet "1" <-- "0..n" Visit

Pet "1" <-- "0..n" Visit

Owner "1" -- "0..n" Pet : owns >

Pet "0..n" --> "0..n" Allergy: has

Allergy "0..n" --> "1..n" ActivePrinciple: has

Medication "0..n" --> "1..n" ActivePrinciple: <font
color=blue>has

Prescription "0..n" --> "1..n" Medication: <font
color=red>mandatory

Prescription "0..n" --> "0..n" Medication: <font
color=red>optional

Prescription "0..n" --> "1" Visit: prescribedIn

%% Style

```

style Pet stroke:blue,color:blue style
ActivePrinciple stroke:blue,color:blue
style Medication stroke:blue,color:blue
style Allergy stroke:red,color:red style
Prescription stroke:red,color:red

```

Las clases para las que realizaremos el mapeo como entidades JPA se han señalado en rojo. Las clases azules son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control. Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias.

- Juego de Ajedrez:

o 1ª Convocatoria (enero) -----

En este ejercicio, añadiremos la funcionalidad de gestión de torneos para una implementación del juego del ajedrez. Concretamente, se proporciona una clase “ChessMatch” que representa las partidas que se juegan, y que tiene asociada una instancia de la clase “ChessBoard” que representa el estado del tablero para dicha partida, por lo que tendrá asociada un conjunto de instancias de la clase “Piece”. Además, tendremos las clases “Tournament”, que representa a un torneo y la clase “Round” que representa a cada una de las rondas de un torneo. Tanto el torneo como las rondas tienen un conjunto de usuarios que son los participantes, que se han expresado en el diagrama con sendas relaciones. El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:

```
classDiagram
```

```
direction LR
```

```

class BaseEntity {
    +Int id
}

```

```
class Authorities {  
    +String authority "NotBlank"  
}
```

```
class User {  
    +String username "Lenght(4,50)"  
    +String password "Lenght(8,50)"  
}
```

```
class NamedEntity {  
    +String name "NotBlank"  
}
```

```
class Round {  
    +String name  
    +Integer roundNumber  
    +LocalDate roundDate  
}
```

```
class Tournament {  
+String name  
    +LocalDate startDate  
    +LocalDate endDate  
    +Integer prize  
}
```

```
class ChessMatch {  
    +LocalDateTime start  
    +LocalDateTime finish
```

```
+ChessMatchType type
+Long turnDuration
}
```

```
class ChessBoard {
+Boolean creatorTurn
+LocalDateTime currentTurnStart
+Boolean jaque
}
```

```
class Piece {
+PieceColor color
+PieceType type
+Integer xPosition
+Integer yPosition
}
```

%% --- Herencia ---

NamedEntity <|-- ChessMatch

BaseEntity <|-- Authorities

BaseEntity <|-- NamedEntity

BaseEntity <|-- Round

BaseEntity <|-- Tournament

%% --- Asociaciones y Relaciones ---

Round "0..1" <-- "1..n" ChessMatch

Round "1..n" --> "1" Tournament

Round "0..n" --> "2..n" User : participants


```

Tournament "0..n" --> "2..n" User: <font
color=red>participants</font>

ChessMatch "1" *--> "1" ChessBoard

ChessMatch "0..n" --> "1" User: <font color=blue>creator</font>

ChessMatch "0..n" --> "1" User: <font color=blue>opponent</font>

ChessBoard "1" *--> "0..n" Piece

Authorities "0..n" -- "" User

```

%% Style

```

style ChessMatch stroke:blue,color:blue

style ChessBoard stroke:blue,color:blue

style Pice stroke:blue,color:blue

style Round stroke:red,color:red

style Tournament stroke:red,color:red

```

Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control de laboratorio. Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias.

o 2ª Convocatoria (julio) -----

En este ejercicio, añadiremos la funcionalidad de gestión de desafíos para una implementación del juego del ajedrez. Concretamente, se proporciona una clase “ChessMatch” que representa las partidas que se juegan, y que tiene asociada una instancia de la clase “ChessBoard” que representa el estado del tablero para dicha partida, por lo que tendrá asociada un conjunto de instancias de la clase “Piece”. Además, tendremos la clase “Challenge”, que representa a un desafío acotado en el tiempo, por ejemplo “ganar 7 partidas en una semana entre el 10 y el 17 de julio de 2025”, o “jugar 100 partidas en el año 2025 (entre el 1 de

enero y el 31 de diciembre)”. Para expresar el objetivo del desafío se usa el enumerado “ChallengeObjective”. Los usuarios del sistema se inscriben en los desafíos (esta situación está expresada a través de la relación “participants”), y cada vez que juegan una partida ésta se asocia con los desafíos en los que estén participando activamente según las fechas y la inscripción (esta situación está expresada a través de la relación entre “Challenge” y “ChessMatch”).

El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:

classDiagram

direction LR

```
class BaseEntity {  
    +Int id  
}
```

```
class Authorities {  
    +String authority "NotBlank"  
}
```

```
class User {  
    +String username "Lenght(4,50)"  
    +String password "Lenght(8,50)"  
}
```

```
class NamedEntity {  
    +String name "NotBlank"  
}
```

```
class Challenge {  
    +String message
```

```
+Int targetValue  
+LocalDate startDate  
+LocalDate endDate  
}
```

```
class ChallengeObjective {  
<<enumeration>>  
    WIN_MATCHES  
    PLAY_MATCHES  
    CAPTURE_PIECES  
}
```

```
class ChessMatch {  
    +LocalDateTime start  
    +LocalDateTime finish  
    +ChessMatchType type  
    +Long turnDuration  
}
```

```
class ChessBoard {  
    +Boolean creatorTurn  
    +LocalDateTime currentTurnStart  
    +Boolean jaque  
}
```

```
class Piece {  
    +PieceColor color  
    +PieceType type  
    +Integer xPosition
```

```
+Integer yPosition
}
```

```
%% --- Herencia ---
```

```
NamedEntity <|-- ChessMatch
```

```
BaseEntity <|-- Authorities
```

```
BaseEntity <|-- NamedEntity
```

```
BaseEntity <|-- Challenge
```

```
%% --- Asociaciones y Relaciones ---
```

```
Challenge "0..n" --> "1..n" ChessMatch
```

```
Challenge "0..n" --> "1" ChallengeObjective
```

```
Challenge "0..n" --> "0..n" User: <font color=red>participant</font>
```

```
ChessMatch "1" *--> "1" ChessBoard
```

```
ChessMatch "0..n" --> "1" User: <font color=blue>creator</font>
```

```
ChessMatch "0..n" --> "1" User: <font color=blue>opponent</font>
```

```
ChessBoard "1" *--> "0..n" Piece
```

```
Authorities "0..n" -- "" User
```

```
%% Style
```

```
style ChessMatch stroke:blue,color:blue
```

```
style ChessBoard stroke:blue,color:blue style
```

```
Pice stroke:blue,color:blue
```

```
style Challenge stroke:red,color:red style
```

```
ChallengeObjective stroke:red,color:red
```

Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas, pero con las que se trabajará durante el control de laboratorio. Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias